cerulean-echoes — Claude-Setup

Was Claude in diesem Projekt immer weiß, und welche Abläufe es auf Abruf kann

1. Aufbau: global und Skills

	`CLAUDE.md`	**Skills** (`.claude/skills/…`)
Wann geladen	in jeder Sitzung automatisch	nur wenn die Aufgabe passt, oder per `/name`
Inhalt	kurze feste Regeln: Ordner, Controller-Belegung, „neue Datei statt überschreiben“, Effizienz-Regeln	ausführliche Abläufe mit Schritten, Befehlen, Zielwerten und Skripten
Länge	kurz halten (kostet in jeder Sitzung Platz)	darf lang sein

Die Skripte der Skills liegen in `tools/` (einmalig `cd tools && npm install`). Ein Hook in `.claude/settings.json` sorgt dafür, dass die Layout-Blätter nach jeder Änderung an einem Track neu erzeugt werden.

Aufrufen: im Chat einfach die Aufgabe beschreiben („mach Track 007 effizienter“, „pegel die Stems“) oder direkt `/strudel-optimize`, `/strudel-mix`, `/lcxl-layout` tippen.

2. `/strudel-optimize` — weniger CPU, gleicher Klang

Problem: Strudel berechnet jedes Event jeder Schicht, auch wenn es stumm ist (Fader auf 0, Density darunter, falscher Obertonmodus). Bei Track 006 waren das 746 Events pro Zyklus; die Rechenzeit lag bei rund 0,7 s pro Zyklus (ein Zyklus dauert 1,7 s) — daher die Aussetzer.

Lösung: stumme Teile werden gar nicht erst abgefragt („Gating“):

```
const gate = (cond, pat) => cond.fmap(on => on ? pat : silence).innerJoin()
```

- **Stems:** `strip()` legt hpf, lpf und Pegel an und schaltet den Stem ab, solange Fader oder Master auf 0 stehen.
- **Layer:** Haupt-/Extra-Layer nur, solange ihr Crossfade-Anteil über 0 ist; Obertöne nur im passenden Modus und oberhalb ihrer Density-Stufe.
- **Buttons und gekoppelte Knöpfe** merken sich ihren Zustand. Sie werden einmal angelegt und über `stateTaps` bei jedem Tick abgefragt, damit kein Tastendruck verloren geht, während ein Stem abgeschaltet ist.
- **Sicherheitsnetz** am Ende: alles mit Gain 0 wird verworfen.

Nachweis: `compare.mjs` spielt alte und neue Fassung mit denselben Reglerstellungen ab (feste Stellungen, Zufallsstellungen, alle Regler in Bewegung, Tastendrücke) und vergleicht jedes hörbare Event.

Track 006 → 006-lean	**Events/Zyklus**	**Rechenzeit/Zyklus**	**identisch**
alle Fader 0	746 → 0	≈ 700 → 65 ms	ja
typisch (Drums + 2 Melodien)	746 → 120	≈ 780 → 170 ms	ja
alles auf 1	746 → 442	≈ 900 → 800 ms	ja

3. `/strudel-mix` — automatisches Pre-Mixing

Jeder Stem wird einzeln offline mit superdough (Strudels Audio-Engine) gerendert und seine Lautheit gemessen (LUFS: Lautheit so, wie das Ohr sie ungefähr empfindet; dazu der Spitzenpegel). Daraus folgen neue Fader-Maxima.

```
node strudel-sim/measure.mjs ../tracks/006-lean-mix.strudel stem1_kick=5,29 stem4_hats=8,32
```

Zielwerte (Faustregel Techno, relativ zur Kick bei Fader 1):

Kick	Sub, Bass	Clap	Melodien	Hats	Perkussion, FX
0 dB	–3 … –5	–5 … –6	–4 … –6	–9 … –11	–8 … –12

Umrechnung: neues Maximum = altes × $10^{dB/20}$ (–3 dB ≈ × 0,71; –6 dB ≈ × 0,5).

Beispiel Track 006: Hats (–13 LUFS) und Clap (–15 LUFS) lagen fast auf Kick-Niveau (–11 LUFS). In `006-lean-mix` sind ihre Fader-Maxima gesenkt: Hats 1 → 0,4, Clap 2,5 → 1,4. Außerdem gemeldet: der Sub übersteuert bei Density 0,5 (Spitzenpegel 3,3).

Grenzen: LUFS unterschätzt helle, kurze Klänge (Hats, Claps), und kleine Lautsprecher verlieren den Bass. Die Messung liefert die Grobeinstellung — den letzten Schliff macht das Ohr.

4. `/lcxl-layout` — Layout-Blätter, die nicht veralten

Das Layout-Blatt (PDF) entsteht aus Kommentaren direkt im Track, neben dem Code, der den Regler liest. Die CC-Nummern rechnet das Skript selbst aus der Position aus.

```
// @col 16:4 4 Hats
// @ctl 16:4.k1 HPF | 300–8000 Hz
// @ctl 16:4.fader Level | 0–0.4
const hatsCol = col(4)
```

`@layout`, `@note`	Titel und Einleitungstext
`@mode 16 …`	eine Seite pro Mode
`@col`, `@row`	Spaltenüberschrift, Zeilenbeschriftung (\ = Zeilenumbruch)
`@ctl <pos> Name | Text`	ein Regler. `pos` = `16:1.k1` (Mode:Spalte.Reihe); Reihen `k1 k2 k3 fader b1 b2`
`2:1-7.k1`	gleicher Regler in mehreren Spalten; ab Spalte 2 steht „wie …“
`g.fader`	globale Spalte 8, auf jeder Seite, CC n / n + 48

Automatisch aktuell:

- Wenn Claude einen Track ändert, erzeugt der Hook das PDF sofort neu; kaputte Tags werden gemeldet.
- Bei Änderungen von Hand: `cd tools && npm run watch` läuft im Hintergrund und erzeugt bei jedem Speichern neu.
- Alle auf einmal: `npm run layouts`.

5. Dateien

`CLAUDE.md`	globale Projektregeln
`.claude/skills/*/SKILL.md`	die drei Skills
`.claude/settings.json`	Hook für die Layout-Blätter
`tools/strudel-sim/`	`harness` (Track in Node auswerten), `compare` (Gleichheit + CPU), `render` + `measure` (Lautheit)
`tools/lcxl-layout/`	`gen-layout` (Tags → PDF), `lcxl3.typ` (Vorlage), `hook`, `watch`
`docs/skills.typ`	dieses Dokument